

✓ Sintaxe básica e Variaveis

Nessa aula, iremos aprender a sintaxe básica de python. Também apresentaremos as variáveis numéricas e booleanas e as suas respectivas operações.

Considerações

No python não se tem uma função `main` como em C++, que seria a função que é seguida quando o arquivo é executado (é possível fazer algo do genero, mas não é obrigatório, vamos falar mais para frente no curso). Com isso, os seus scrips são seguidos linha por linha. Sem a necessidade de uma função principal.

Além disso, o Python possui tipagem dinâmica, ou seja, não precisamos falar qual o tipo da nossa variavel, ele identifica e ajusta dinamicamente.

Variaveis numéricas

São aquelas variaceis que representam um número (escalar). No python temos 3, `int` para números inteiros, `float` para numeros racionais, `complex` para numeros complexos.

```
numero = 1          # int
numero2 = 3.1415     # float
numero3 = 2.3 + 4.6j # complex
```

No python, não se preocupamos muito com a memoria que cada variavel guarda, já que o isso é ajustado dinamicamente. Mas, a fim de curiosidade, o `int` é ilimitado, se ajustando conforme necessário. Já o float tem um tamanho fixo, limitado pela memória do sistema, com um valor máximo aproximado de 1.8×10^{308} .

Input and Output (User IO)

Para enviar dados para o usuário, podemos utiliza o `print()` no python. Ele vai publicar no terminal o que enviarmos para ele. Veja abaixo:

```
a = 10
print(a)
>> 10
```

Podemos também printar varias informações por vez:

```
a = 15
print("Este é meu número", a, ", muito bom.")
>> Este é meu número 15 , muito bom.
```

```
1 # Operadores Aritiméticos
2
3 # dado duas variaveis inteiras temos:
4 a = 18
5 b = 5
6
7 # Adição
8 soma = a + b
9
10 # Subtração
11 sub = a - b
12
13 # Multiplicação
14 multi = a * b
15
16 # Divisão
17 div = a / b
18
19 # Divisão (parte inteira)
20 div_int = a // b
21
22 # Resto
```

```

23 resto = a % b
24
25 # Potenciação
26 pot = a ** b
27
28 # Printando os resultados no terminal
29 print(soma)
30 print(sub)
31 print(multi)
32 print(div)
33 print(div_int)
34 print(resto)
35 print(pot)

```

```

23
13
90
3.6
3
3
1889568

```

Podemos observar algo muito interessante, e característico da diferença de Python e C++. Na divisão de 18 por 5 resulta em um número não inteiro, e como pode se observar o python já reconhece que se trata de um número racional (float). Podemos confirmar isso utilizando a função `type()` em python, que retorna o tipo da variável.

```

1 print(type(a))
2 print(type(b))
3 print(type(div))

```

```

<class 'int'>
<class 'int'>
<class 'float'>

```

Essas operações podem ser utilizadas pros outros tipos de variáveis numéricas. Além disso, pode-se misturar as variáveis numéricas, o python vai se ajustando sozinho.

```

1 c = 3.1415 # float
2 d = 3.2 + 8j # complex
3
4 # Operações multi variavel
5 e = a + c + d
6 f = (a/c) + (d * b)
7
8 print(e)
9 print(type(e))
10
11 print(f)
12 print(type(f))

```

```

(24.3415+8j)
<class 'complex'>
(21.729746936176987+40j)
<class 'complex'>

```

Existem também os operadores de atribuição. Eles servem para simplificar o código evitando esse tipo de escrita:

```
a = a + b
```

Se reescreve usando operadores de atribuição:

```
a += b
```

Os operadores de atribuição que existem são:

```

1 a = 18.6
2
3 # Adição
4 a += 5
5 print(a)
6

```

```
7 # Subtração
8 a -= 2
9 print(a)
10
11 # Multiplicação
12 a *= 2
13 print(a)
14
15 # Divisão
16 a /= 4
17 print(a)
```

```
23.6
21.6
43.2
10.8
```

Importante: Note que os operadores de atribuição são sempre o sinal da operação antes do símbolo de igual. Se você inverter a ordem, pode estar sem querer atribuindo o valor ao invés de adicionar ou subtrair. Veja o exemplo abaixo:

```
1 a = 18.6
2
3 # Jeito Certo
4 a += 5
5 print(a)
6
7 # Jeito Errado
8 a =+ 5
9 print(a)
```

```
23.6
5
```

```
1 a = 18.6
2
3 # Jeito Certo
4 a -= 5
5 print(a)
6
7 # Jeito Errado
8 a =- 5
9 print(a)
```

```
13.600000000000001
-5
```

✓ Conversão de tipos

Como comentado, o python faz a conversão automática dos tipos de variável. Mas muitas vezes queremos um tipo específico. Ai podemos utilizar os conversores do python.

```
1 # Conversão float -> int
2 a = 15.6
3 b = int(a)
4
5 print(b)
6 print(type(b))
7
8 # Conversão int -> float
9 a = 10
10 b = float(a)
11
12 print(b)
13 print(type(b))
14
15 # Conversão int -> complex
16 a = 10
17 b = complex(a)
18
19 print(b)
20 print(type(b))
```

```
15
<class 'int'>
10.0
<class 'float'>
(10+0j)
<class 'complex'>
```

✓ Analisando a Variavel Complexa

A variavel complexa ela tem as peculiaridades de um número complexo, então não há uma conversão direta dela pra float ou int. Contudo, podemos acessar as partes reais e imaginaria do número:

```
1 # Analisando o número complexo
2 a = 10 + 5j
3
4 print(a.real)
5 print(a.imag)
6 print(a.conjugate())
7
8 # Convertendo para float
9 b = float(a.real)
10 print(b)
11 print(type(b))
12
13 # Convertendo pra int
14 c = int(a.real)
15 print(c)
16 print(type(c))
```

```
10.0
5.0
(10-5j)
10.0
<class 'float'>
10
<class 'int'>
```

✓ Coletando Input do usuário

Para coletar dados do terminal fornecidos pelo usuário, pode se usar o `input()`:

```
result = input("Insira um valor:")
```

```
1 result = input("Insira um valor:")
2 print(result)
3 print(type(result))
```

```
Insira um valor:2
2
<class 'str'>
```

Podemos observar que o tipo do input é um string (`str`), com isso não podemos fazer operação numérica com ela. Com isso vamos converter a string lida para um numerico. Vamos estudar string também mais a frente no curso.

```
1 input_a = int(input("Insira um valor:"))
2 input_b = int(input("Insira outro valor:"))
3
4 result = input_a + input_b
5 print(result)
```

```
Insira um valor:4
Insira outro valor:8
12
```

✓ Variaveis Booleanas

Caso nunca tenham ouvido falar de variaveis ou de logica booleana, ela se trata de operações de sim e não, de `True` e `False`. As variaveis só podem assumir dois estados.

Operações

Podemos fazer multiplas operações com logica booleana e esse assunto rende um semestre inteiro como Sistemas Digitais, já que é o principio dos computadores modernos.

Operações Lógicas

Os operadores lógicos primordiais são:

- NOT: Operação "NÃO", inverte o valor fornecido;
- AND: Operação "E", retorna `True` somente se ambos forem `True`;
- OR: Operação "OU", retorna `True` se qualquer uma das entradas forem `True`;

```
1 # Criando variaveis booleanas
2 a = True
3 b = False
4
5 # Operações
6 print("Operação NOT")
7 print(not True)
8 print(not False)
9
10 print("Operação E")
11 print(True and True)
12 print(True and False)
13 print(False and True)
14 print(False and False)
15
16 print("Operação OU")
17 print(False or True)
18 print(True or False)
19 print(True or True)
20 print(False or False)
21
```

```
Operação NOT
False
True
Operação E
True
False
False
False
Operação OU
True
True
True
False
```

Operadores Relacionais

Além dos operadores lógicos comuns, temos os operadores relacionais. Esses operadores também podem ser chamados de operadores comparativos. Eles realizam a comparação de valores e variaveis, são eles:

- ">", maior que
- "<", menor que
- "==" , igual a
- "!=" , diferente de

```

1 a = 18.6
2 b = 5
3
4 print("A é maior que B?")
5 print(a > b)
6
7 print("B é menor que A?")
8 print(b < a)
9
10 print("A é igual a B?")
11 print(a == b)
12
13 print("A é diferente de B?")
14 print(a != b)

```

```

A é maior que B?
True
B é menor que A?
True
A é igual a B?
False
A é diferente de B?
True

```

Importante: Note que os operadores de igual é dois símbolos de igualdade. Se você usar somente um, não vai estar comparando e sim atribuindo um valor.

```

1 a = 18.6
2 b = 5
3
4 # Jeito Certo
5 is_equal = (a == b)
6 print(is_equal)
7
8 # Jeito Errado
9 is_equal = (a = b)
10 print()

```

```

File "/tmp/ipython-input-348600955.py", line 9
  is_equal = (a = b)
               ^

```

SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?

Importante: Para o símbolo de diferente, a ordem correta é o "!=" antes do igual. Se você fizer ao contrário, irá atribuir o valor contrário da variável;

```

1 a = 18.6
2 b = 5
3
4 # Jeito Certo
5 is_not_equal = (a != b)
6 print(is_not_equal)
7
8 # Jeito Errado
9 is_not_equal = (a =! b)
10 print(is_not_equal)

```

```

File "/tmp/ipython-input-207054761.py", line 9
  is_not_equal = (a =! b)
                  ^

```

SyntaxError: invalid syntax

Como pode-se observar, o interpretador de python identifica esse tipo de erro na maioria dos casos. Porém é sempre bom verificar.

Exemplos de usos

```

1 # Calculando a área de um retângulo dados os inputs do usuário
2 a = float(input("Insira o valor da base:"))

```

```

3 b = float(input("Insira o valor da altura:"))
4
5 area = a * b
6
7 print(area)

```

```

Insira o valor da base:15.8
Insira o valor da altura:14.9
235.42000000000002

```

```

1 # Comparador de áreas de retangulo dado seus lados
2
3 # Calculando a área de um retângulo dados os inputs do usuário
4 b1 = float(input("Insira o valor da base do retângulo 1:"))
5 h1 = float(input("Insira o valor da altura do retângulo 1:"))
6 areal = b1 * h1
7
8 b2 = float(input("Insira o valor da base do retângulo 2:"))
9 h2 = float(input("Insira o valor da altura do retângulo 2:"))
10 area2 = b2 * h2
11
12 print("Area do retângulo 1 é maior que a área do retângulo 2")
13 print(area)

```

▼ Importando bibliotecas no python

Nem todas as funções que precisamos utilizar estão disponíveis na linguagem pura de Python. Por isso, contamos com módulos Python, as bibliotecas que contém códigos que possam nos ajudar e que já contém funções uteis. Vamos ver como importamos esses módulos.

Para importar bibliotecas no python, é muito simples, só digitar import e o nome da biblioteca:

```
import library
```

Mas podemos apelidar as bibliotecas, caso no nome seja muito grande ou se repita muito:

```
import library as lib
```

Podemos também escolher o que importar, passando o caminho de onde está:

```
from library import my_func, MyClass
```

Veja, que podemos passar uma lista de coisas que queremos importar.

```

1 # Importando a biblioteca math do python
2 import math
3
4 a = 4
5 print(math.sqrt(4))

```

```
2.0
```

Veja que tivemos que escrever `math.sqrt()` pra chamar a função `sqrt`. Precisamos utilizar o "." para descrever que aquela função está dentro da biblioteca math e não é outra função chamada "sqrt". Caso não queira utilizar a `math.funcao()` você pode importar a função direto também:

```

1 from math import sqrt, sin, cos
2
3 a = sin(math.pi/2) # 90 graus
4 b = 16
5 print("Temos", sqrt(b), "e", a)

```

```
Temos 4.0 e 1.0
```

E podemos combinar:

```

1 from math import sqrt as raiz, sin as seno, cos as cosseno
2
3 a = seno(math.pi/2) # 90 graus

```

```
4 b = 16
5 print("Temos", raiz(b), "e", a)
```

Temos 4.0 e 1.0

Ajeitar pra ficar mais visual:

```
1 from math import (
2     sqrt as raiz,
3     sin as seno,
4     cos as cosseno
5 )
6
7 a = seno(math.pi/2) # 90 graus
8 b = 16
9 print("Temos", raiz(b), "e", a)
```

Temos 4.0 e 1.0